

Project Synopsis (Draft) – Parallel Computing (PCAP)

GPU-Accelerated Graph Traversal: BFS and SSSP with Load-Balancing Strategies using CUDA

Team: Diya, Niharika, Sanjay

1. Problem Statement

Graph algorithms such as BFS (Breadth-First Search) and SSSP (Single-Source Shortest Path) are core operations in search engines, social networks, navigation and recommendation systems. On graphs with millions of vertices and edges, a sequential CPU implementation becomes slow because each step must process the neighbour lists of a large and constantly changing set of active vertices (the frontier).

Because the edge operations within a step are independent, the workload is naturally data-parallel and suited to a GPU. However, real graphs are highly irregular: a few hub vertices have very large numbers of connections while most have very few. A naive GPU mapping that assigns one thread per vertex therefore causes severe **load imbalance** — most threads finish instantly and remain idle while a few overloaded threads dominate the runtime. This project implements BFS and SSSP on the GPU using CUDA and studies how different work-mapping (load-balancing) strategies affect performance on graphs with different structures.

2. Objectives

- Implement correct **sequential CPU baselines** for BFS and SSSP using the CSR graph format.
- Profile the baselines and confirm that frontier/neighbour processing is the dominant bottleneck.
- Implement **CUDA GPU versions** of BFS and SSSP, verified against the CPU baselines.
- Implement and compare **three load-balancing strategies**: thread-per-vertex, warp/block-per-vertex, and edge-based (frontier) mapping.
- Evaluate **execution time, speedup, parallel efficiency, TEPS and load imbalance** across graph types and sizes; optionally extend to multi-GPU using MPI.

3. General Methodology

Pipeline: Input edge list → convert to CSR format → sequential CPU baseline (correctness reference) → profiling / bottleneck identification → CUDA GPU implementation → add load-balancing strategies → correctness verification against CPU → benchmarking and performance analysis.

- **Graph representation:** CSR (Compressed Sparse Row) — row offsets + column indices; compact and gives efficient, coalesced neighbour access on the GPU.
- **BFS:** level-synchronous frontier expansion — each level expands the current frontier to its neighbours.
- **SSSP:** iterative edge relaxation (Bellman-Ford / delta-stepping style) until distances converge.
- **What is parallelized:** the per-step processing of the frontier's outgoing edges, which are independent within a step (data parallelism).
- **Correctness:** GPU BFS levels and SSSP distances must exactly match the CPU baseline output.

4. Parallel Programming Models – What is Used Where

Model	Where it is used	Why
CUDA (primary)	The main GPU implementation of BFS and SSSP, and all three load-balancing strategies (thread-per-vertex, warp/block-per-vertex, edge-based mapping).	The workload is millions of small, identical, independent edge operations per step and is memory-bound — exactly what a GPU's thousands of cores and high memory bandwidth are designed for. OpenMP's few CPU cores cannot provide this level of fine-grained parallelism.
OpenMP (baseline)	A multi-threaded CPU version of BFS and SSSP, used as the comparison baseline alongside the single-threaded version.	Ensures a fair CPU-vs-GPU comparison: the GPU is compared against a properly parallelised multicore CPU implementation, not only against single-threaded code. Shared-memory model, simple to apply to the existing CPU baseline.
MPI (optional extension)	Only in the stretch goal: partitioning a graph across 2 GPUs / 2 processes with boundary (halo) exchange between partitions.	MPI is a distributed-memory model, needed only when a graph is too large for a single GPU's memory. Our selected graphs fit on one GPU, so MPI's communication overhead is unnecessary for the core project and is kept as an optional extension.

Summary: CUDA does the actual acceleration, OpenMP provides a fair CPU baseline for comparison, and MPI is used only if we scale beyond a single GPU.

5. Performance Evaluation

- **Execution time:** sequential CPU vs OpenMP CPU vs each GPU strategy.
- **Speedup** = $T(\text{serial}) / T(\text{parallel})$; **Parallel efficiency** = Speedup / P.
- **TEPS** (Traversed Edges Per Second) — the standard graph-traversal throughput metric.
- **Load imbalance** across the three strategies, and GPU occupancy / memory bandwidth via Nsight.
- **Scalability:** performance as graph size grows (and across GPUs, if the MPI extension is done).

6. Datasets / Input

Dataset	Approx. size	Type / why chosen
SNAP soc-LiveJournal1	~4.8M vertices, ~69M edges	High-degree social graph (power-law) — where load imbalance is worst.
SNAP com-Orkut	~3M vertices, ~117M edges	Dense social graph; stresses memory bandwidth and frontier size.
SNAP roadNet-CA	~2M vertices, ~5.5M edges	Low-degree, near-uniform road network — contrast case where naive mapping already balances well.
Graph500 (Kronecker generator)	Configurable	Synthetic graphs of controllable size for the scalability study.

All datasets are publicly available. Format: plain-text edge lists, converted once into CSR. The contrast between high-degree social graphs and low-degree road networks is what makes the load-balancing comparison meaningful.

7. References

- [1] Y. Wang et al., “Gunrock: GPU graph analytics,” *ACM Trans. Parallel Comput.*, vol. 4, no. 1, 2017.
- [2] M. Osama, S. D. Porumbescu and J. D. Owens, “A programming model for GPU load balancing,” *Proc. ACM PPOPP*, 2023.
- [3] C. Yang, A. Buluç and J. D. Owens, “GraphBLAST: A high-performance linear algebra-based graph framework on the GPU,” *ACM Trans. Math. Softw.*, vol. 48, no. 1, 2022.
- [4] H. Yang et al., “XBFS on AMD GPUs,” *SC24 Workshops*, 2024.
- [5] “Towards efficient load-balancing BFS on GPUs: one code for AMD, Intel and Nvidia,” *SC’25 Workshops*, 2025.
- [6] R. Bhoot, S. S. Ghanmode and Y. Simmhan, “TARIS: Scalable incremental processing of time-respecting algorithms on streaming graphs,” *IEEE Trans. Parallel Distrib. Syst.*, 2024.
- [7] J. Leskovec and A. Krevl, “SNAP Datasets: Stanford large network dataset collection,” 2014. [Online]. Available: <http://snap.stanford.edu/data>
- [8] Graph500 benchmark. [Online]. Available: <https://graph500.org>

Note: verify full author lists and page numbers against IEEE Xplore / ACM DL when preparing the final report; the course requires expanding this to 8–10 papers.

8. Division of Work – 3 Members

Member	Algorithm / technical ownership	Research & report
Niharika BFS (CPU + GPU)	Sequential CPU BFS (queue-based) and OpenMP CPU BFS; CUDA BFS kernel with level-synchronous frontier expansion; implements the three load-balancing strategies for BFS; correctness check of GPU BFS levels against the CPU baseline.	BFS-related literature (Gunrock, XBFS, SC’25 BFS load balancing); writes the BFS algorithm and methodology sections.
Diya SSSP (CPU + GPU)	Sequential CPU SSSP (Dijkstra / Bellman-Ford) and OpenMP CPU SSSP; CUDA SSSP kernel using edge relaxation (delta-stepping style); implements the three load-balancing strategies for SSSP; handles atomics and race-condition-free distance updates; correctness check of GPU distances against the CPU baseline.	SSSP / relaxation and GPU load-balancing literature; writes the SSSP algorithm and methodology sections.
Sanjay Performance analysis of all methods	Builds the shared benchmarking and timing harness; profiles all versions with gprof and NVIDIA Nsight; measures and compares every algorithm × every strategy (thread-per-vertex, warp/block-per-vertex, edge/frontier-based) across all datasets; computes speedup, efficiency, TEPS, load imbalance and occupancy; produces all graphs; implements the optional MPI multi-GPU extension.	Performance-analysis and comparison literature; writes the experimental setup, results and performance-analysis sections.

Shared components: the CSR data pipeline and the load-balancing strategy definitions are agreed jointly so that BFS and SSSP use the same three strategies, making Sanjay's cross-algorithm comparison fair. All three members contribute code, research and presentation content.

9. Expected Outcome and Contribution

We do not propose a new algorithm. Our contribution is a correct, from-scratch GPU implementation of BFS and SSSP together with a systematic comparison of load-balancing strategies, showing which strategy performs best on which type of graph (high-degree social networks vs low-degree road networks) and identifying where performance is limited by memory bandwidth or residual load imbalance.